

Σavante

Sistemas de Gestión de Bases de Datos relacionales, orientados a objetos y NoSQL: características y componentes

ÍNDICE

1. Sistema de bases de datos	3
1.1. Bases de datos (B.D.)	4
1.2. Sistema Gestor de Bases de Datos (SGBD)	6
1.2.1. Objetivos de un SGBD	6
1.2.2. Transacciones de los SGBD	7
2. Modelos principales de B.D.	8
3. Sistemas de Gestión de B.D. Relacionales	10
3.1. El modelo relacional	11
3.2. Conceptos fundamentales	12
3.3. Integridad de los datos	15
3.4. Reglas de Codd	15
3.5. Lenguajes de Codd para manipulación de los datos	19
3.5.1. Álgebra relacional	19
3.5.1.1. Básicas (operador fundamental)	21
3.5.1.2. No básicas o Derivadas	23
3.5.2. Cálculo relacional	26
3.5.3. Diferencia entre álgebra y cálculo relacional	26
3.6. Principales SGBDR	26
4. Sistemas de Gestión de B.D. No relacionales (NoSQL)	30
4.1. Tipos de B.D. NoSQL	30
4.1.1. Bases de datos Clave-Valor	31
4.1.2. Bases de datos documentales	31
4.1.3. Bases de datos de grafos	33
5. Sistemas de Gestión de B.D. Orientados a objetos	34
5.1. Manifiesto del Sistema de Base de Datos Orientado a Objetos	35
5.2. Principales SGBDOO	36
6. Comparativa entre tipos de B.D.	42
6.1. Entre las BDOO con las BDR	43
6.2. Entre B.D (NoSQL) y las relacionales	44
7. Bibliografía	44



1. Sistema de bases de datos



Fuente: Pixabay



Recuerda ver las clases emitidas en Temario Audiovisual

Las clases impartidas en directo y disponibles en Campus, en Temario Audiovisual te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)

Un sistema de Bases de datos, está formado por varios componentes, que vas a estudiar en esta unidad.

Los componentes son:

- Una **base de datos** (puede ser de diferentes tipos).
- Un **Sistema Gestor de Bases de datos** que administra y gestiona la información de la base de datos.
- Un **diccionario de datos**. Contiene el listado de campos y variables de la BD, así como su descripción, longitud, posibles valores, etc.



También puede contener otros datos de interés como:

- Información sobre la representación física de los datos.
 - Asignación a dispositivos.
 - Formas de acceso.
 - Índices.
- **Administrador de bases de datos (DBA).** Persona o grupo de personas que crean, gestionan y mantienen la Base de Datos.
 - **Usuario.** Son aquellas personas que utilizan la Base de Datos.

1.1. Bases de datos (B.D.)

Existen múltiples definiciones de bases de datos.

Una de las más aceptadas es la propuesta por Flory en 1982:

*Una **base de datos** es un conjunto exhaustivo, no redundante de datos estructurados, organizados independientemente de su utilización y su implementación en máquina, accesibles a tiempo real y compatibles por usuarios concurrentes que tienen necesidad de información diferente y no predecible en el tiempo.*

Sin embargo, existen muchas otras entre las que podríamos destacar las siguientes:

- Conjunto, colección o depósito de datos almacenados en un soporte informático. Los datos deben estar interrelacionados y estructurados de acuerdo con un modelo capaz de recoger el máximo contenido semántico.
- Consiste en una colección de datos persistentes e independientes usados por una organización determinada.
- Serie de datos organizados y relacionados entre sí, los cuales son recolectados y explotados por los sistemas de información de una empresa o negocio en particular.
- Es un conjunto de datos pertenecientes a un mismo contexto y almacenados sistemáticamente para su posterior uso.
- Es una colección de información organizada de tal modo que sea fácilmente accesible, gestionada y actualizada.
- Es una representación de objetos y situaciones del mundo real. En el mundo real existen restricciones y limitaciones que deben ser reflejadas en la base de datos. Para ello es necesario el uso de métodos de diseño rigurosos y formalizados.



Características

En base a estas definiciones, podemos definir las características principales de una base de datos:

- Es un conjunto o colección de datos.
- Los datos están estructurados.
- Existen relaciones entre los datos.
- Los datos no pueden ser redundantes (no debe haber duplicados).
- Los datos deben ser independientes de la máquina en que se almacenan o explotan.
- Debe ser fácilmente accesible, gestionada y almacenada.
- Debe permitir el acceso concurrente a la misma (de esto se encargan los Sistemas Gestores de Base de Datos (SGBD)).
- Deben dar soporte a usuarios con distintas necesidades.
- La mayoría de las bases de datos se almacenan en un soporte informático.
- Son la base de los sistemas de información.
- Representan una situación del mundo real.

Niveles de abstracción de una base de datos

Dividimos la abstracción de un dato en 3 niveles:

- **Nivel Interno:** describe la estructura física de almacenamiento de la base de datos, todos los detalles para su almacenamiento, y los caminos de acceso para acceder al dato. (Es el nivel más cercano al almacenamiento físico).
- **Nivel Externo (o de vistas):** describe la parte de la base de datos que interesa a un grupo de usuarios determinado, como esos usuarios ven los datos, y les oculta el resto de la base de datos. Incluye diferentes vistas de usuario o esquemas. (Es el nivel más cercano a los usuarios, como ven los datos).
- **Nivel Conceptual:** tiene un esquema conceptual que describe la estructura de la base de datos, pero oculta los detalles físicos del almacenamiento. Describir tipos de datos, entidades, vínculos, restricciones y operaciones de los usuarios. (Es un nivel de mediación entre el interno y el externo).



1.2. Sistema Gestor de Bases de Datos (SGBD)

Un Sistema Gestor de Bases de Datos (SGBD) es un conjunto coordinado de programas, procedimientos, lenguajes, etc. que suministra tanto a los usuarios como al administrador de la base de datos, los medios necesarios para describir, manipular y utilizar los datos almacenados en la base, manteniendo la integridad, confidencialidad y seguridad. Su objetivo principal es simplificar y facilitar el acceso a datos.

1.2.1. Objetivos de un SGBD

Los principales objetivos de un sistema de base de datos son:

- **Proporcionar a los usuarios y desarrolladores una visión abstracta de los datos.**

El sistema esconde ciertos detalles de cómo se almacenan y mantienen los datos.

- **Independencia entre datos y aplicaciones.**

Cuando se tenga que cambiar algo en la base de datos (como la forma de almacenar datos), esto no repercutirá en los programas de aplicación que trabajan sobre esa base de datos.

Evitar redundancias de datos (duplicidad de información).

- **Evitar la inconsistencia de datos.**

Para que haya inconsistencia debe haber datos redundantes.

Se produce una inconsistencia de datos cuando existen varias copias de un mismo dato y no todas tienen el mismo valor.

- **Preservar la integridad.**

Los valores de los datos almacenados deben satisfacer ciertos tipos de restricciones de consistencia.

Por ejemplo, el precio de un producto debe ser un número y no una cadena de caracteres.

- **Atomicidad.**

Los procesos deben ser atómicos, es decir, deben ocurrir o no ocurrir, pero no puede ocurrir parte del proceso.

Por ejemplo, si mientras se está realizando un proceso se produce un corte de electricidad y se apaga el equipo, Se deberá volver al estado de consistencia anterior al fallo para que no se queden operaciones a medio hacer.



- **Seguridad y Confidencialidad.**

Se debe garantizar la confidencialidad y seguridad de los datos contra accesos incorrectos o no autorizados.

- **Acceso concurrente a los datos.**

Debe permitir a múltiples usuarios actualizar los datos simultáneamente.

1.2.2. Transacciones de los SGBD

Una transacción es una interacción con una estructura de datos compleja, compuesta por varios procesos que se han de aplicar uno después del otro.

La transacción debe realizarse de una sola vez y sin que la estructura a medio manipular pueda ser alcanzada por el resto del sistema hasta que se hayan finalizado todos sus procesos.

En bases de datos se denomina ACID a las características de los parámetros que permiten clasificar las transacciones de los sistemas de gestión de bases de datos.



+ Info

ACID es un acrónimo de Atomicity, Consistency, Isolation and Durability:

En español: Atomicidad, Consistencia, Aislamiento y Durabilidad.

Cuando se dice que es ACID compliant se indica -en diversos grados- que éste permite realizar transacciones.

Propiedades de las transacciones ACID

- **Atomicity** (Atomicidad). Si cuando una operación consiste en una serie de pasos, de los que o bien se ejecutan todos o ninguno, es decir, las transacciones son completas.
- **Consistency** (Consistencia, Integridad). Es la propiedad que asegura que sólo se empieza aquello que se puede acabar. Por lo tanto, se ejecutan aquellas operaciones que no van a romper las reglas y directrices de Integridad de la base de datos. La propiedad de consistencia sostiene que cualquier transacción llevará a la base de datos desde un estado válido a otro también válido. "La Integridad de la Base de Datos nos permite asegurar que los datos son exactos y consistentes, es decir que estén siempre intactos, sean siempre los esperados y que de ninguna manera cambian ni se deformen. De esta manera podemos garantizar que la información que se presenta al usuario será siempre la misma."



- **Isolation** (Aislamiento). Esta propiedad asegura que una operación no puede afectar a otras. Esto asegura que, la realización, de dos transacciones sobre la misma información sean independientes, y no generen ningún tipo de error. Esta propiedad define cómo y cuándo los cambios producidos por una operación se hacen visibles para las demás operaciones concurrentes. El aislamiento puede alcanzarse en distintos niveles, siendo el parámetro esencial a la hora de seleccionar SGBDs.
- **Durability** (Durabilidad o Persistencia). Esta propiedad asegura que, una vez realizada la operación, esta persistirá y no se podrá deshacer, aunque falle el sistema y que de esta forma los datos sobrevivan de alguna manera.

2. Modelos principales de B.D.

Las B.D. se pueden clasificar de acuerdo a su modelo de administración de datos.

Un modelo de datos es básicamente una "descripción" de algo conocido como contenedor de datos (algo en donde se guardan los datos), así como de los métodos para almacenar y recuperar datos de esos contenedores.

Los modelos de datos no son cosas físicas: son abstracciones que permiten la implementación de un sistema eficiente de base de datos; por lo general se refieren a algoritmos, y conceptos matemáticos.

En función a las diferentes formas de administración de datos, podemos clasificar las bases de datos en distintos tipos, los siguientes los estudiamos en esta unidad:

- **B.D. Relacionales (SGBDR).**
- **B.D. No Relacionales (NoSQL).**
 - Bases de datos documentales.
 - Bases de datos de grafos.
- **B.D. Orientadas a Objetos (SGBDOO).**

Indicamos también otros tipos importantes:

- **Bases de datos jerárquicas.**

La información se almacena en una estructura jerárquica, la cual enlaza los registros existentes en forma de una estructura de árbol, donde el nodo padre (sección principal de la información) puede tener varios nodos hijos, es decir, el modelo jerárquico facilita las relaciones hijo-padre.

El sistema gestor no se encarga del control sobre los datos que se almacenan, si no que se encargan las aplicaciones gestoras, lo que ocasiona diversos problemas como la duplicidad de registros entre otros.



- **Bases de datos de red.**

También llamadas bases de datos plex, son similares al modelo anterior, pero en este caso un mismo nodo puede tener varios padres, lo cual no se puede hacer en el modelo jerárquico. Así se soluciona algunas deficiencias del modelo de bases de datos jerárquicas como por ejemplo relaciones de muchos a muchos.

- **Bases de datos transaccionales.**

Se utilizan para los sistemas en que los datos cambian constantemente, y ofrecen datos en tiempo real. Son bastante rápidas, y el típico ejemplo es para el uso de transferencias bancarias.

No tienen problemas de duplicidad, desnormalización o redundancia en la información, si no que aseguran la integridad de los datos con modificaciones seguras sin poner en riesgo la integridad de los datos.

- **Bases de datos multidimensionales.**

Son similares a las bases de datos relacionales, pero aquí un atributo de una tabla puede ser de dos tipos o representar dimensiones en dicha tabla. Eso supone el inconveniente de que una vez creada la base de datos y diseñada su estructura no es posible modificarla.

Este tipo de bases de datos son adecuadas para análisis de negocios, gestión avanzado de datos y minería de datos, ya que son muy eficientes en consultas manejando los datos de forma rápida y eficaz (más que las bases de datos relacionales). Por ejemplo, para la creación de cubos OLAP (OnLine Analytical Processing), donde los datos físicos son almacenados en un vector multidimensional.

- **Bases de datos deductivas.**

También se les llama bases de datos lógicas, ya que se basan en la lógica matemática, permitiendo realizar deducciones a través hipótesis.

Para responder a las consultas se realizan deducciones basándose en los hechos almacenados en la base de datos.

Se utiliza una fase de interrogación para buscar informaciones deducibles y después una fase de modificación para agregar a la base de datos esas nuevas informaciones deducibles.

Utilizan el lenguaje declarativo Datalog (subconjunto del lenguaje de programación Prolog).

Surgieron para mejorar las limitaciones de las bases de datos relacionales ante consultas recursivas.

- **Bases de datos geoespaciales.**

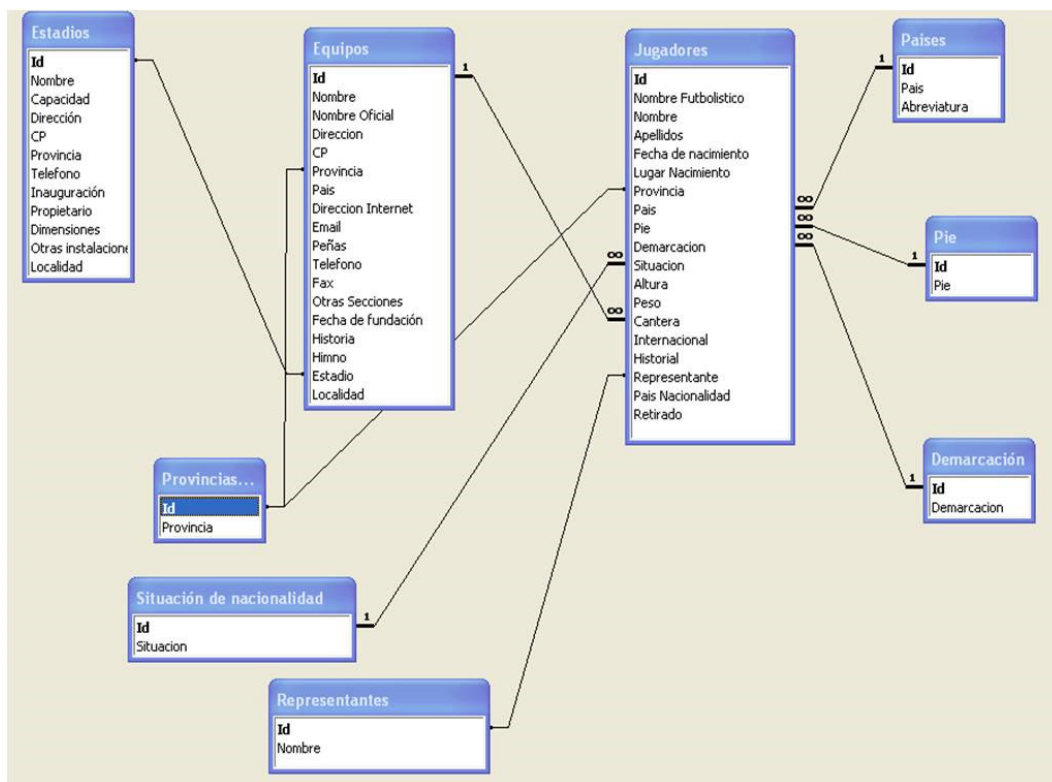
Estas bases de datos se originan por la necesidad de disponer de un modelo para tratar la información geoespacial, almacenando los datos geográficos y sus relaciones espaciales y no espaciales, así como atributos y comportamiento.



Cada Geo-objeto tiene sus propios atributos, comportamiento y geometría, por lo que la base de datos puede trabajar con cada uno de ellos de forma casi independiente.

Cuando se analizan los datos, la terminología no es datos o líneas, etc., sino que se habla de lugares específicos (montes, lagos, carreteras, etc.).

3. Sistemas de Gestión de B.D. Relacionales



Modelo_relacional de wikimedia Commons

Codd propuso un modelo simple de datos en el que todos ellos se podían representar en tablas constituidas por filas y columnas. A estas tablas se les dio el nombre de relaciones (de aquí viene el nombre de modelo relacional).

Ambos lenguajes soportan la manipulación de los datos sobre la base de operadores lógicos en lugar de los punteros físicos utilizados en los modelos jerárquico y en red.

En los sistemas de bases de datos relacionales, los archivos completos de datos se pueden procesar con instrucciones sencillas (en los tradicionales se debían procesar registro a registro).

SQL (Structured Query Language) es la norma ANSI para los lenguajes relacionales de bases de datos.



Definiciones de base de datos relacional

Una base de datos relacional es un tipo de base de datos que permite establecer interconexiones o relaciones entre los datos organizados en tablas. A través de dichas conexiones, relacionan los datos de las tablas.

Otra definición más informal sería:

Una base de datos relacional es una base de datos en donde todos los datos visibles al usuario están organizados estrictamente como tablas de valores, y en donde todas las operaciones de la base de datos operan sobre estas tablas.

Sin embargo, la definición de un SGBDR más aceptada es:

Aquel SGBD que cumpla las 12 reglas de Codd.



+ Info

Codd también propuso dos lenguajes para manipular los datos en las tablas:

- El álgebra relacional.
- El cálculo relacional.

Vamos a estudiarlos más adelante.

3.1. El modelo relacional



Básico

Edgar Frank Codd postulo las bases del modelo relacional en 1970.



En una base de datos relacional, los usuarios la perciben como un conjunto de tablas.

Ejemplo:

Una tabla que contenga la información de artículos en venta se vería así:

ProductId	NombreProducto	PrecioEuros
0001	ratón	3
0002	teclado	6

Tabla "Articulos".

En el modelo relacional se utiliza un grupo de tablas para representar datos y las relaciones entre ellos.

A cada tabla se le asigna un nombre exclusivo.

Su idea fundamental es el uso de relaciones. Las relaciones se conceptualizan como si fuera una tabla.

La representación obtenida del modelo entidad-relación, es independiente de las características del equipo donde se vaya a implantar.

3.2. Conceptos fundamentales

Tabla

Objeto que contiene una colección de datos para un tema específico. Las tablas constan de filas y columnas.

Columna

Componente vertical de una tabla de base de datos. Una columna tiene un nombre y un tipo de datos específico, por ejemplo, carácter, decimal o entero.

Fila

Componente horizontal de una tabla, que consta de una secuencia de valores, uno para cada columna de la tabla.



Vista

Una vista es una consulta que se presenta como una tabla (virtual) a partir de un conjunto de tablas en una base de datos relacional.

Las vistas tienen la misma estructura que una tabla: filas y columnas.

En una vista, sólo se almacena de ellas la definición, no los datos, que se recuperan de la tabla donde estén almacenados, mediante una consulta, (una sentencia SELECT) que la calcula, y que puede realizarse sobre una o más tablas. Sobre un conjunto de tablas relacionales se puede trabajar con un número cualquiera de vistas.

Igual que en una tabla, a través de una vista, se pueden insertar, actualizar, borrar y seleccionar datos en una vista. En ocasiones existen restricciones de forma que sólo podemos seleccionar datos de una vista, pero no realizar el resto de las operaciones sobre vistas.

La mayoría de los SGBD soportan la creación y manipulación de vistas. Las vistas se crean cuando se necesitan hacer varias sentencias para devolver una tabla final.

Tupla

Una tupla se define como una función finita que asocia unívocamente los nombres de los campos de una relación con los valores de una instancia de la misma.

En término más sencillo, es una fila de una tabla relacional. También se le denomina Instancia o registro.

(Es cada una de las ocurrencias de la entidad representada por la tabla).

Cardinalidad

La Cardinalidad se entiende como la relación entre tablas de una base de datos, es un valor constante ya que preve la evolución de las mismas.

Atributo

Es una columna de la tabla (equivale a un campo de una tupla).

Grado

El número de atributos se llama grado. (también se le puede llamar paridad).

El grado no varía con el tiempo. Si añadimos un atributo a una relación, podemos considerar que se trata de otra relación nueva.



Dominio

Es una colección de valores, de los cuales uno o más atributos obtienen sus valores reales.

Clave (o llave) candidata

Es un atributo (o conjunto de atributos) de una relación R que cumple las siguientes propiedades:

- **Unicidad:** no existen dos tuplas con el mismo valor en el atributo o conjunto de atributos.
- **Minimalidad:** si es un atributo compuesto (conjunto de atributos), no se puede eliminar ningún componente sin destruir la propiedad de unicidad.
- Si se elimina un atributo de una clave candidata, está deja de serlo.

Clave (o llave) primaria

Si una relación (tabla) tiene más de una clave candidata, se escoge una de ellas como primaria y el resto pasan a ser claves alternativas.

Clave foránea (ajena o extranjera)

Es un atributo (o conjunto de atributos) de una relación (tabla) que es clave primaria de otra relación (tabla).

Se utiliza para referenciar a la tupla de la otra relación cuya clave primaria coincida con el valor de la clave foránea.

Ambas claves deben definirse sobre el mismo dominio.



Atención

En el modelo Entidad/Relación extendido, según Métrica v3 las relaciones se definen por:

Cardinalidad, nombre, y tipo de correspondencia.

(Lo estudiaremos en profundidad en el Módulo III).



3.3. Integridad de los datos

En una base de datos relacional, las relaciones están sujetas a ciertas reglas de integridad:

- **Integridad de entidad.**

Ningún atributo que participe en una clave principal puede tener valores nulos.

- **Integridad referencial.**

No deben existir valores de clave ajena sin concordancia.

Esto quiere decir que un determinado valor de una clave ajena siempre tiene que aparecer en la clave primaria de alguna tupla de la relación referenciada.

3.4. Reglas de Codd

A continuación, vamos a estudiar las 12+1 reglas de Codd que debe cumplir una base de datos para ser completamente relacional.



+ Info

No pienses que escribimos 12+1 porque somos supersticiosos.

Codd definió doce reglas para identificar bases de datos relacionales.

Luego añadió una denominada "Regla 0" para identificar cuando un sistema gestor de bases de datos (SGBD) es relacional (SGBDR).

Codd define doce reglas que debe obedecer una base de datos para ser relacional.

- **Regla 1: Regla de la información.**

Toda la información en una base de datos relacionales se muestra explícitamente en el nivel lógico mediante tablas y solo mediante tablas.

Aclaración: Los metadatos (diccionario, catálogo) se representan y manipulan exactamente igual que los datos de usuario.



- **Regla 2: Regla de acceso garantizado.**

Para todos y cada uno de los datos (valores atómicos) de una base de datos relacional se garantiza que son accesibles a nivel lógico utilizando una combinación de nombre de tabla, valor de clave primaria y nombre de columna.

Aclaración: Cualquier dato almacenado en una base de datos relacionales tiene que poder ser direccionado unívocamente indicando la tabla, columna y fila.

- **Regla 3: Tratamiento sistemático de valores nulos.**

Los valores nulos (no confundir con cadena vacía, cadena de caracteres en blanco, 0, etc.) se soportan en los SGBD totalmente relacionales para representar información desconocida o no aplicable de manera sistemática, independientemente del tipo de datos.

Aclaración: Se reconoce la necesidad de la existencia del valor nulo, para representar una información desconocida. Por ejemplo, un valor nulo en el teléfono no indica que no tenga teléfono, sino que lo desconocemos.

- **Regla 4: Catálogo dinámico en línea basado en el modelo relacional.**

La descripción de la base de datos se representa a nivel lógico de la misma manera que los datos normales, de modo que los usuarios autorizados pueden aplicar el mismo lenguaje relacional a su consulta, igual que lo aplican a los datos normales.

Aclaración: Los metadatos se almacenan y se manejan usando el modelo relacional.

- **Regla 5: La regla comprensiva del sub-lenguaje de los datos.**

Un sistema relacional puede soportar varios lenguajes y varios modos de uso de terminal (por ejemplo, rellenar formularios). El sistema debe soportar por lo menos un lenguaje relacional que:

- Tenga una sintaxis lineal.
- Puede ser utilizado de manera interactiva.
- Tenga soporte de operaciones de definición de datos, operaciones de manipulación de datos (actualización, así como la recuperación), de control de la seguridad e integridad y operaciones de administración de transacciones.

Sin embargo, debe haber al menos un lenguaje cuyas sentencias sean:

- Expresables mediante alguna sintaxis bien definida, como cadenas de caracteres.
- Completas en cuanto al soporte de todos los puntos siguientes:
 - » Definición de datos.
 - » Definición de vista.



- » Manipulación de datos (interactiva y por programa).
- » Restricciones de integridad.
- » Autorización.
- » Fronteras de transacciones (comienzo, cumplimiento y vuelta atrás).

Aclaración: Aunque un SGBD ofrezca interfaces amigables para el usuario o administrador, se debe disponer de un lenguaje relacional que pueda realizar, al menos, las funciones que se pueden realizar desde los interfaces.

- **Regla 6: Regla de actualización de vistas.**

Todas las vistas que son teóricamente actualizables deben ser actualizables por el sistema.

Aclaración:

- Las vistas son tablas virtuales utilizadas para dar, a diferentes usuarios, distintas vistas de su estructura.
 - Es una de las reglas más difíciles de implementar en la práctica, ya que cada sistema puede hacer las suposiciones particulares sobre las vistas que son actualizable.
- **Regla 7: Alto nivel de Inserción, actualización y eliminación.**

La capacidad de manejar una relación base o derivada como un solo operando se aplica no sólo a la recuperación de los datos (consultas), sino también a la inserción, actualización y eliminación de datos.

Aclaración:

- Requiere que las filas sean tratadas como conjuntos en operaciones de inserción, supresión y actualización.
 - Por ejemplo, si quiero eliminar todas las filas que cumplen una determinada condición, no tengo que ir una a una, sino que puedo eliminarlas todas con una instrucción.
- **Regla 8. Independencia física de los datos.**

Los programas de aplicación y actividades del terminal permanecen inalterados a nivel lógico cualesquiera que sean los cambios efectuados (tanto en la representación del almacenamiento, como en los métodos de acceso).

Aclaración:

- El modelo relacional es un modelo lógico de datos, y oculta las características de su representación física.
- Se aísla el programa de aplicación de la implementación a bajo nivel de los datos.



- **Regla 9: Independencia lógica de los datos.**

Los programas de aplicación y actividades del terminal permanecen inalterados a nivel lógico cualesquiera que sean los cambios que se realicen sobre las tablas que contienen la información.

Aclaración: De nuevo, se aísla el programa de aplicación de la implementación a bajo nivel de los datos.

- **Regla 10: Independencia de la integridad.**

Las limitaciones de la integridad se deben especificar por separado de los programas de la aplicación y se almacenan en la base de datos. Debe ser posible cambiar esas limitaciones sin afectar a las aplicaciones que trabajan sobre la base de datos.

Aclaración: El objetivo de las bases de datos no es sólo almacenar los datos, sino también sus relaciones y restricciones para evitar que esto se tenga que codificar en los programas.

Deben tener las siguientes reglas de integridad:

- Integridad de Entidad: Toda tabla debe tener una clave primaria.
- Integridad de Dominio: Toda columna de una tabla contendrá valores exclusivamente de un determinado dominio (conjunto de valores válidos).
- Integridad Referencial: Toda clave foránea no nula debe existir en la relación donde es clave primaria.

- **Regla 11: Independencia de la distribución.**

Una Base de Datos Relacional es independiente de la distribución.

Aclaración: La distribución de la base de datos debe ser invisible a los usuarios. Se debe trabajar igual en una base de datos centralizada que en una distribuida.

- **Regla 12: La regla de la no subversión.**

Si un sistema relacional tiene un lenguaje de bajo nivel, el nivel bajo no puede ser usado para subvertir (saltarse) las reglas de integridad y las restricciones expresadas en los lenguajes relacionales de más alto nivel.

Aclaración: Normalmente se usa SQL para solucionar problemas que no se pueden desde el lenguaje de alto nivel. Usando SQL no debe ser posible saltarse las restricciones de integridad.

Es muy difícil que una Base de Datos cumpla todas las reglas. Sin embargo, se realizan pruebas para ver cuantas reglas cumple y eso sirve para definir el grado en que se puede considerar una base de datos relacional (será más relacional cuantas más reglas cumpla).



Codd añadió una regla más (Regla 0) para determinar si un SGBD es relacional. Es la siguiente:

- **Regla 0:**

Para que un sistema se denomine sistema de gestión de bases de datos relacionales, este sistema debe usar (exclusivamente) sus capacidades relacionales para gestionar la base de datos.

Por ejemplo, para añadir un usuario se debe poder utilizar el mismo lenguaje que se usa para añadir un registro a una tabla.



Técnicas de estudio

Mi consejo es que hagas un resumen.

Utiliza una sola línea para definir cada regla.

Esa línea debe ayudarte a comprenderla y a recordarla.

3.5. Lenguajes de Codd para manipulación de los datos

Codd también propuso dos lenguajes para manipular los datos en las tablas:

- El álgebra relacional.
- El cálculo relacional.

Vamos a estudiar el álgebra relacional y el cálculo relacional, y veremos sus diferencias.

3.5.1. Álgebra relacional

Los lenguajes de procedimientos para consultar bases de datos relacionales están basados en el álgebra relacional.

El Álgebra Relacional fue introducida por E. F. Codd en 1972.



El álgebra relacional es un conjunto de operaciones, que describen paso a paso cómo computar una respuesta sobre las relaciones, tal y como éstas son definidas en el modelo relacional.

Describe el aspecto de la manipulación de datos. Estas operaciones se usan como una representación intermedia de una consulta a una base de datos y, debido a sus propiedades algebraicas, sirven para obtener una versión más optimizada y eficiente de dicha consulta.



Resumiendo

El Álgebra Relacional consiste en un conjunto de operaciones sobre las relaciones.

Las operaciones son:

- Básicas (operador fundamental):
 - Selección - restricción (σ).
 - Proyección (Π).
 - Producto cartesiano ($\times$).
 - Unión ($\cup$).
 - Diferencia ($-$).
- No básicas o Derivadas:
 - Intersección $\cap$.
 - Unión natural ($\bowtie$) (Natural Join).
 - División ($\bar{\div}$) (Cociente).
 - Agrupación (γ) (Unión).

Vas a estudiarlas una a una, con gráficos para que te resulte más fácil.



3.5.1.1. Básicas (operador fundamental)

SELECCIÓN o SELECT: Restricción (σ)

Extrae tuplas a partir de una relación que satisfagan una restricción dada.

Por ejemplo, si utilizamos un SELECT con la condición $\text{precio} = 5$, nos mostraría todas las tuplas de la tabla cuyo atributo "precio" tenga el valor 5.

PROYECCIÓN o PROJECT (II)

Selecciona ciertos atributos (columnas) de la relación original y elimina las tuplas duplicadas.

Veamos un ejemplo:

IDVENTA	NOMBRECOMPRADOR	CIUDAD
1	Pepe	Córdoba
2	Manuel	Zaragoza
3	María	Zaragoza
4	Andrés	Madrid

Si hacemos una proyección sobre esta tabla para extraer el atributo "ciudad" nos quedará:

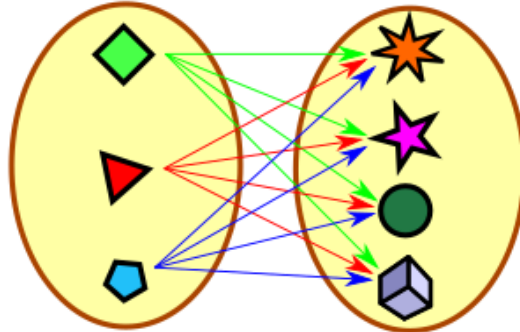
CIUDAD
Córdoba
Zaragoza
Madrid

Como se puede observar, se han eliminado las tuplas duplicadas (dos con el valor "Zaragoza").

PRODUCTO CARTESIANO ($\times$)

No es necesario que las 2 relaciones tengan la misma cabecera.

El producto cartesiano de dos relaciones nos devuelve una relación con las tuplas resultantes de combinar cada fila de la primera relación con todas las filas de la segunda relación.

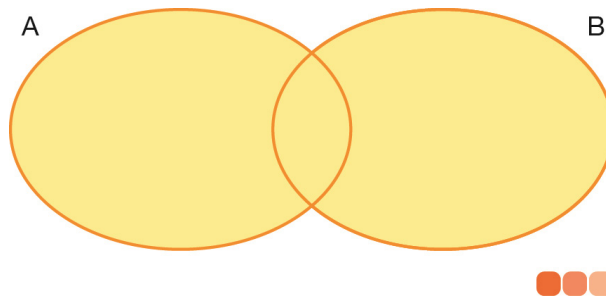


Producto cartesiano (fuente:
https://es.wikipedia.org/wiki/Archivo:Producto_cartesiano_de_conjuntos.svg)

UNIÓN (U)

El operador de unión toma dos relaciones que deben ser compatibles para la unión (tener la misma cabecera, es decir, los mismos atributos y en el mismo orden).

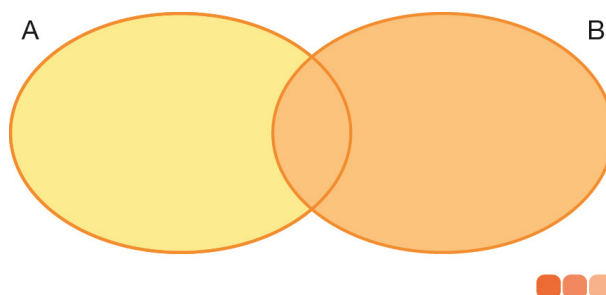
La relación resultante tendrá la misma cabecera y todas las tuplas de ambas relaciones (eliminando duplicados).



Unión de dos conjuntos

DIFERENCIA (-)

La diferencia de dos relaciones con la misma cabecera es una relación con la misma cabecera formada por el conjunto de todas las tuplas que están en la primera relación y no están en la segunda.



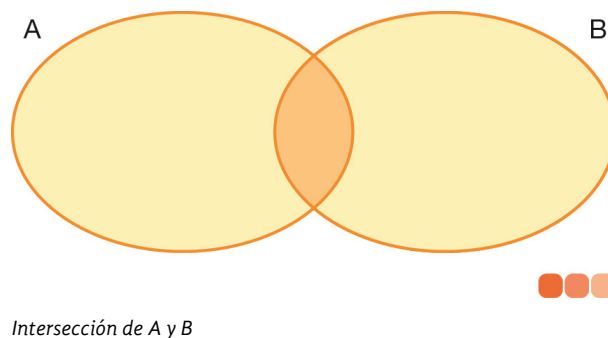
Diferencia de A y B



3.5.1.2. No básicas o Derivadas

INTERSECCIÓN $\cap$

A partir de dos relaciones del mismo tipo, (con la misma cabecera), mediante la intersección se obtiene una nueva relación formada por las tuplas que pertenecen a las dos relaciones de partida.



Unión natural ($\bowtie$) (Natural Join)

La operación unión natural en el álgebra relacional es la que permite reconstruir las tablas originales previas al proceso de normalización.

Consiste en combinar las proyección, selección y producto cartesiano en una sola operación, donde la condición θ es la igualdad Clave Primaria = Clave Externa (o Foránea), y la proyección elimina la columna duplicada (clave externa).

Expresada en las operaciones básicas, queda:

$$R \bowtie S = \Pi_{A_1, A_2, \dots, A_n} (\sigma_{\theta} (R \times S))$$

Una reunión theta (θ -Join) de dos relaciones es equivalente a:

$$R \bowtie_{\theta} S = \sigma_{\theta} (R \times S)$$

Donde la condición θ es libre.

Si la condición θ es una igualdad se denomina EquiJoin.



DIVISIÓN ($\div$) (Cociente)

Dada una relación "relacion1" con los atributos A, B, C y D y otra relación "relacion2" con los atributos C y D:

La división relacion1/relacion2 será una relación con los atributos que no están en la segunda relación (A y B) y cuyas tuplas serán las tuplas de relacion1 cuyos valores de C y D estén en la relación 2.

Por si te has liado un poco, vamos a ver un ejemplo para aclararlo:

Relacion1.

A	B	C	D
1	2	3	4
1	2	5	6
2	3	5	6
5	4	3	4
5	4	5	6
1	2	4	5

Relacion2.

C	D
3	4
5	6

- Paso1. Eliminamos las tuplas de relacion1 donde C y D no coinciden con ninguna tupla de relacion2.

Resultado.

A	B	C	D
1	2	3	4
1	2	5	6



A	B	C	D
2	3	5	6
5	4	3	4
5	4	5	6

Hemos borrado la última tupla porque la tupla C=4 y D=5 no está en relacion2.

- Paso 2. Eliminamos las columnas que están en relacion2 (C y D).

Resultado.

A	B
1	2
1	2
2	3
5	4
5	4

- Paso 3. Eliminamos duplicados.

Resultado.

A	B
1	2
2	3
5	4

Agrupación (γ) (Unión)

Permite agrupar conjuntos de valores en función de un campo determinado y hacer operaciones con otros campos.



3.5.2. Cálculo relacional

El Cálculo relacional es un lenguaje de consulta que describe la respuesta deseada sobre una Base de datos sin especificar como obtenerla.

El Cálculo Relacional se basa en la lógica de primer orden. Hay dos variantes del cálculo relacional:

- **Cálculo Relacional de Dominios (DRC).** Las variables son atributos de las tuplas.
- **El Cálculo Relacional de Tuplas (TRC).** Las variables son tuplas.

Cuantificadores

Los cuantificadores son símbolos que se utilizan para expresar la frecuencia con la que una propiedad o relación se cumple en una relación.

$\forall$ (cuantificador universal) se utiliza para expresar que una propiedad o relación se cumple para todos los elementos de una relación.

$\exists$ (cuantificador existencial) se utiliza para expresar que una propiedad o relación se cumple para al menos un elemento de una relación.

$\exists!$ (cuantificador único) se utiliza para expresar que una propiedad o relación se cumple para exactamente un elemento de una relación.

3.5.3. Diferencia entre álgebra y cálculo relacional

El Álgebra relacional es de tipo procedimental mientras que el cálculo relacional es de tipo declarativo.

3.6. Principales SGBDR

Vamos a ver un resumen de las características de las principales SGBDR que son:

- MySQL.
- Oracle.
- Microsoft SQL Server.
- PostgreSQL.
- Microsoft Access.



MySQL

El experto opina

En nuestra opinión es el mejor SGBD gratuito (aunque requiere licencia para usarlo con fines privativos).

Es el SGBD ideal para iniciarse.

Puedes probarlo junto a PHPMyAdmin. Es una buena combinación.

Es la base de datos de código abierto más conocida.

Es un sistema de gestión de base de datos relacional multihilo y multiusuario.

Se ofrece bajo la GNU GPL, pero si una empresa quiere utilizarlo con fines privativos, deben adquirir una licencia.

Actualmente pertenece a Oracle.

Utiliza el motor InnoDB.

Ventajas:

- Buena velocidad operacional.
- Pocos requisitos.
- Facilidad de configuración e instalación.

Sentencias principales de MySQL:

- **CREATE DATABASE:** Crear una base de datos en MySQL.
- **ALTER DATABASE:** Modificar una base de datos.
- **DROP DATABASE:** Eliminar una base de datos en MySQL.
- **CREATE TABLE:** de crear una tabla con todas sus características en una base de datos MySQL.

Tras crear una base de datos los tipos de datos que podemos guardar son:

- Numéricos.
- De fecha.
- Tipo string.



- **RENAME TABLE:** renombrar una tabla de una base de datos.
- **TRUNCATE TABLE:** vaciar una tabla.
- **DELETE:** Eliminar filas de tablas.
- **DROP TABLE:** borrar de manera rápida y sencilla una tabla.

Oracle



El experto opina

En nuestra opinión es el mejor SGBDR.

Se aconseja a toda gran empresa, siempre que puedan asumir el coste.

Es un sistema de gestión de base de datos relacional fabricado por Oracle Corporation.

Se considera el SGBDR más robusto y completo (aunque también uno de los más caros).

Oracle ofrece una edición gratuita y de libre distribución (Oracle XE o Express Edition) con muchas limitaciones.

Una tablespace es una unidad lógica de almacenamiento.

Ventajas:

- Soporte de transacciones.
- Estabilidad.
- Escalabilidad.
- Seguridad.
- Es multiplataforma.



Microsoft SQL Server

Utiliza el lenguaje Transact-SQL.

Puede poner a disposición de muchos usuarios grandes cantidades de datos de manera simultánea. (Es un servidor de servicios SQL, el programa que ejecuta las instrucciones de SQL).

Es un sistema propietario de Microsoft.

Tiene una versión Express gratuita y limitada de libre distribución.

Ventajas:

- Soporte de transacciones.
- Escalabilidad.
- Estabilidad.
- Seguridad.
- Soporta procedimientos almacenados.
- Incluye un potente entorno gráfico de administración, que permite el uso de comandos DDL y DML gráficamente.
- Se puede trabajar en modo cliente-servidor.

PostgreSQL

También llamado Postgres. Es un sistema de gestión de bases de datos relacional orientado a objetos y libre, publicado bajo la licencia PostgreSQL, similar a la BSD.

Al ser un proyecto de código abierto, está dirigido por una comunidad de desarrolladores que trabajan de forma desinteresada.

La comunidad PostgreSQL se denominada el PGDG (PostgreSQL Global Development Group).

Ventajas:

- Alta concurrencia.
- Amplia variedad de tipos nativos.
- Es gratuito.
- Estabilidad.
- Confiabilidad.
- Seguridad.



Microsoft Access

Es un SGBDR creado por Microsoft para uso personal o de pequeñas organizaciones.

No se debe usar para bases de datos grandes.

Ventajas:

- Fácil de utilizar.
- Tiene herramientas para el desarrollo de aplicaciones sobre la base de datos utilizando VBA.
- Puede realizar llamadas a APIs de Windows.
- Permite realizar consultas, formularios e informes.

4. Sistemas de Gestión de B.D. No relacionales (NoSQL)

Con la llegada de aplicaciones web como Facebook, YouTube o Twitter, todo el mundo empezó a subir contenidos a la red (en forma de textos, videos, imágenes, etc.).

Las bases de datos SQL no estaban preparadas para tratar ni el tipo ni la cantidad de datos que se maneja en internet.

Apareció un nuevo modelo para cubrir estas necesidades: Sistemas de BD no Relacionales (NoSQL).

NoSQL significa Not Only SQL (no solo SQL).



4.1. Tipos de B.D. NoSQL

Este tipo de Bases de Datos, no utilizan el modelo relacional.

Hay muchas bases de datos NoSQL y aún no hay una estandarización como sí que la hay en las relacionales.



4.1.1. Bases de datos Clave-Valor

Son las más populares y sencillas de entender, ya que cada elemento está identificado por una llave única, lo que permite la recuperación de la información de forma muy rápida buscando por su clave. Esto hace que sean eficientes tanto en lectura como escritura.

Vamos a ver algunas de las principales bases de datos Clave-Valor:

- **Riak:**

Traducción del inglés-Riak es un almacén de datos de valor clave NoSQL distribuido que ofrece alta disponibilidad, tolerancia a fallas, simplicidad operativa y escalabilidad. Además de la versión de código abierto, viene en una versión empresarial compatible y una versión de almacenamiento en la nube.

- **DynamoDB:**

Es un servicio de base de datos NoSQL que ofrece Amazon dentro de sus Amazon Web Services.

- **Redis:**

Se ha creado en C. Es de código abierto.

Funciona en Linux, Unix y OS X, pero no hay soporte oficial para Windows.

- **Cassandra:**

Creada por Apache, dispone de un lenguaje propio para consultas (CQL o Cassandra Query Language).

Se ha desarrollado en Java, por lo que puede ejecutarse sobre cualquier plataforma que tenga la máquina virtual de Java.

4.1.2. Bases de datos documentales

La información se almacena como un documento utilizando algún formato estándar como JSON o XML.

Se utiliza una clave única para cada documento.

Se pueden hacer búsquedas por clave-valor, pero también otras más avanzadas sobre el contenido del documento.

Su diseño está pensado para almacenar grandes volúmenes de información, normalmente datos históricos.

Destacan dos características principales: indexación a texto completo, para hacer posible que las búsquedas en ese gran volumen de datos sean eficientes y el hecho de que los documentos pueden direccionarse por medio de claves únicas (normalmente cadenas especiales y en algún caso especial una URL).



Si comparamos con las bases de datos relacionales, los documentos son similares a las filas. , pero aquí no es necesario que todos los documentos tengan los mismos atributos por lo que este tipo de bases de datos son muy flexibles.

Aunque se pueden organizar los documentos de forma libre, hay unas determinadas formas que son las más utilizadas como la organización mediante etiquetas o colecciones, metadatos ocultos y jerarquías de directorios.

Algunos de los sistemas de administración de bases de datos documentales más importantes son:

- **ArangoDB:** base de datos multi-modelo que combina los modelos Clave/Valor, Documentos y Grafos en un solo núcleo desarrollado en C++. Utiliza el lenguaje de consultas, AQL (ArangoDB Query Language).
- **BaseX:** especializado en almacenar, consultar y visualizar colecciones y documentos XML de gran tamaño.
- **Apache CouchDB** (nombrado normalmente como CouchDB):

CouchDB se liberó en 2005, y se transformándose en un proyecto Apache en 2008, convirtiéndose en código abierto. Es una base de datos NoSQL que utiliza JSON para guardar los datos y JavaScript de lenguaje de consulta. Destacas la facilidad con la que permite hacer replications.

- **MongoDB:**

Es una de las más populares, de código abierto y escrito en C++.

En lugar de guardar los datos en tablas como se hace en las bases de datos relacionales, MongoDB utiliza para almacenar la información un sistema propio llamado BSON.

BSON es una evolución de JSON que puede almacenar datos binarios, es por tanto un formato de intercambio de datos, con una representación binaria de estructuras de datos y mapas.

El nombre BSON significa Binary JSON (JSON Binario).



+ Info

JSON (JavaScript Object Notation) es un formato de texto ligero para el intercambio de datos.



4.1.3. Bases de datos de grafos

Están basadas en la teoría de grafos. La información se representa como nodos de un grafo y sus relaciones como las aristas.

Para tener un mejor rendimiento, la estructura debe estar normalizada (cada tabla debe tener una sola columna y cada relación dos columnas).

Son especialmente útiles en modelos con muchas relaciones (como las redes sociales).

Algunas de las principales bases de datos de grafos son:

- **ArangoDB:**

Base de datos multi-modelo que combina los modelos Clave-Valor, Documentos y Grafos en un sólo núcleo desarrollado en C++. Utiliza el lenguaje de consultas AQL (ArangoDB Query Language).

- **Infinite Graph:**

Pertenece a la compañía Objectivity y tiene dos tipos de licencia (gratuita y de pago).

Está escrita en Java y C++.

- **Neo4j:**

Es de código abierto, escrita en Java. Pertenece a la compañía Neo Technology.



Atención

Cypher.

Fue en gran parte una invención de Andrés Taylor mientras trabajaba para Neo4j.

Es un lenguaje de consulta (sobre B.D. Neo4j) de gráfico declarativo que permite realizar consultas de datos expresivas y eficientes en un gráfico de propiedades.



5. Sistemas de Gestión de B.D. Orientados a objetos

Un SGBDOO es un sistema que combina características de orientación a objetos y lenguajes de programación OO (orientado a objetos) con capacidades de bases de datos [Martínez, 1996].

En un SGBDOO, los datos aparecen en forma de objetos, permitiendo a determinados lenguajes de programación verlos también como objetos.

Un SGBDOO otorga a los lenguajes varias capacidades:

- La posibilidad de tener datos persistentes de una forma transparente.
- Control de concurrencia.
- Recuperación de datos.
- Consultas asociativas.

Las bases de datos orientadas a objetos se diseñan para trabajar bien en conjunción con lenguajes de programación orientados a objetos como Java, C#, Visual Basic.NET y C++ ya que usan el mismo modelo.



+ Info

Las bases de datos orientadas a objetos, en realidad, también podrían considerarse bases de datos NoSQL.

¿Por qué surgen?

Los sistemas de gestión de bases de datos orientadas a objetos surgen debido a la falta de capacidad semántica del Modelo Relacional para atender nuevos tipos de aplicaciones como son:

- Diseño en ingeniería (CASE, CAD/ CAM).
- Bases de datos gráficas y de imágenes.
- Bases de datos científicas.
- Sistemas de información geográfica.
- Bases de datos multimedia.



5.1. Manifiesto del Sistema de Base de Datos Orientado a Objetos

El Manifiesto del Sistema de Base de Datos Orientado al Objeto (The Object-Oriented Database System Manifesto) fue desarrollado por Atkinson, Bancilhon, DeWitt, Maier y Zdonik.

Este manifiesto del Sistema de Base de Datos Orientado a Objetos, indica que características debe tener una base de datos para ser orientada a objetos (SGBDOO).

Son las siguientes características, que se dividen en tres categorías:

- Obligatorias.
- Optativas.
- Abiertas.

Obligatorias

Debe satisfacer dos criterios fundamentales:

- Debe ser un SGBD. Por lo tanto, debe tener las siguientes características:
 - Persistencia.
 - Gestión de almacenamiento secundario.
 - Recuperación.
 - Concurrencia.
 - Facilidad de consultas Ad-hoc.
- Debe ser un sistema orientado a objetos, por lo que debe contemplar los siguientes conceptos:
 - Objetos complejos.
 - Identidad de objetos.
 - Encapsulamiento.
 - Tipos y Clases.
 - Herencia.
 - Sobrecarga.
 - Extensibilidad.
 - Completitud computacional.



Optativas

Son características que debería cumplir, pero no es necesario implementarlas.

Estas son:

- Herencia múltiple.
- Comprobación de tipos.
- Distribución.
- Transacciones de Diseño.
- Versiones.

Abiertas

- Paradigma de programación.
- Sistema de representación.
- Sistema de tipos.
- Uniformidad.

5.2. Principales SGBDOO

A continuación presentamos varios Sistemas de Gestión de Bases de Datos Orientadas a Objetos (SGBDOO) están ordenados prioritariamente por uso actual y secundariamente por relevancia histórica:

- InterSystems Caché.
- ObjectDB.
- Zope Object Database (ZODB).
- OpenLink Virtuoso.
- Objectivity/DB.
- Versant Object Database.
- DB4O.



- ObjectStore.
- Perst.
- Matisse.
- ConceptBase.
- OpenAccess.
- PicoLisp.
- FastObjects.
- Ozone.
- ObjectDatabase++.
- Jasmine.

InterSystems Caché

InterSystems Caché es un sistema comercial de administración de bases de datos operacionales de InterSystems, que se utiliza para desarrollar aplicaciones de software para administración de servicios de salud, servicios bancarios y financieros, gobierno y otros sectores.

El software del cliente puede usar base de datos orientada a objetos o código SQL.

ObjectDB

ObjectDB es un SGBDOO está estrechamente integrado con Java.

Se puede usar en modo cliente-servidor.

Lo que distingue a ObjectDB de otras bases de datos orientadas a objetos es su enfoque exclusivo en el almacenamiento persistente de objetos Java de manera directa, sin necesidad de usar mapeo objeto-relacional (ORM). Esto la hace muy eficiente en aplicaciones Java, donde se puede almacenar directamente objetos complejos (incluyendo relaciones entre objetos) en su forma nativa.

Soporta los estándares JPA (Java Persistence API) y JDO (Java Data Objects), facilitando su integración en aplicaciones Java.

Ofrece un alto rendimiento y escalabilidad, optimizada para el almacenamiento eficiente de objetos complejos y estructuras jerárquicas. A través de JPQL y JDOQL, permite realizar consultas similares a SQL.



Zope Object Database

La base de datos orientada a objetos Zope (ZODB) es una base de datos orientada a objetos utilizada para almacenar objetos de Python de forma transparente y persistente.

Se incluye como parte del servidor de aplicaciones web de Zope, pero también se puede usar independientemente.

Las características de ZODB son:

- Transacciones.
- Historial / deshacer.
- Almacenamiento transparente.
- Almacenamiento en caché integrado.
- Control de concurrencia multi-versión (MVCC).
- Escalabilidad en una red (usando ZEO).

Openlink Virtuoso

Virtuoso es un híbrido de middleware y motor de base de datos que combina varias funcionalidades:

- Sistema de gestión de bases de datos relacionales.
- Base de datos relacional orientada a objetos (ORDBMS).
- Base de datos virtual.
- RDF (para representación de grafos).
- XML.
- Texto libre.
- Servidor de aplicaciones web.
- Servidor de archivos.

Virtuoso es un "servidor universal" que permite un único proceso de servidor multiproceso que implementa múltiples protocolos.

La edición gratuita y de código abierto de Virtuoso Universal Server también se conoce como OpenLink Virtuoso.

El software ha sido desarrollado por OpenLink Software.



+ Info

Es un sistema muy polivalente.

Se puede utilizar, por ejemplo, como un SGBDR, como un SGBDOO y como SGBD NoSQL para almacenar grafos.

Objectivity/DB

Objectivity/DB es un SGBDOO comercial creado por Objectivity, Inc.

Permite a las aplicaciones almacenar objetos estándar de C++, C#, Java o Python.

Objectivity/DB es compatible con los lenguajes más populares orientados a objetos, con SQL, ODBC y XML.

Se ejecuta en plataformas Linux, Macintosh, UNIX y Windows.

Versant Object Database

Versant Object Database (VOD) es un producto software para bases de datos orientadas a objetos desarrollado por Versant Corporation.

La base de datos orientada a objetos de Versant permite a los desarrolladores que utilizan lenguajes orientados a objetos almacenar su información de forma transaccional al permitir que dicho lenguaje actúe como el lenguaje de definición de datos (DDL) para la base de datos.

DB4O

db4o (database for objects) es un sistema gestor de base de datos orientado a objetos de código abierto para desarrolladores Java y .NET.

ObjectStore

ObjectStore es un SGBD comercial.

Está inspirado en la base de datos Statice desarrollada originalmente en Symbolics.

ObjectStore es innovador en el uso del lenguaje C ++ para hacer que el acceso a la base de datos sea transparente.



Los objetos se pueden crear en una base de datos al sobrecargar el operador `new ()`.

De esta manera, se puede almacenar objetos de C++ directamente en la base de datos y estos objetos persistentes se ven y se comportan como los objetos normales de C++.

Perst

SGBDOO con doble licencia, código abierto y comercial. Es ligero y eficiente, diseñado para almacenar objetos de manera persistente en aplicaciones Java y .NET sin la necesidad de mapeo objeto-relacional (ORM). Su bajo consumo de recursos y alto rendimiento lo hacen ideal para aplicaciones con restricciones de memoria y procesamiento, mientras que su API sencilla facilita su integración en proyectos. Al ser multiplataforma, permite su uso en distintos entornos, lo que lo convierte en una opción atractiva para desarrolladores que buscan una solución rápida y eficiente para manejar datos complejos en aplicaciones Java y .NET.

Tanto la versión de Java como la de C# son pequeñas, por lo que se ha podido implementar en smartphones con sistemas operativos Android y Windows.

Matisse

SGBDOO diseñado para manejar datos complejos y estructuras jerárquicas, combinando las ventajas del paradigma orientado a objetos con un enfoque escalable. Es utilizado principalmente en sectores como sistemas de información geográfica (SIG), diseño asistido por computadora (CAD) y aplicaciones científicas. Integra de forma nativa características como herencia, encapsulación y polimorfismo, facilitando su uso con lenguajes de programación orientados a objetos. Está específicamente diseñado para integrarse de forma estrecha con el lenguaje de programación Smalltalk.

Soporta herencia, polimorfismo y encapsulación directamente en la base de datos.

Persistencia transparente: Permite almacenar objetos sin necesidad de mapeo relacional.

Es compatible con Java, C++ y otros lenguajes orientados a objetos.

Matisse opera bajo un modelo de licencia comercial, adecuada para aplicaciones empresariales y proyectos específicos.

ConceptBase

ConceptBase es un sistema de gestión de base de datos deductivo y orientado a objetos desarrollado en la Universidad de Aachen y la Universidad de Skövde.

Se utiliza principalmente para modelado conceptual y metamodelado en el campo de la ingeniería de software y campos relacionados.

ConceptBase es un software gratuito y de código abierto.



OpenAccess

OpenAccess es una API propietaria que pertenece a OpenAccess Coalition.

Tienen como objetivo facilitar la interoperabilidad del software de automatización de diseño electrónico entre los miembros de esa coalición.

Una de las características más distintivas de OpenAccess es su capacidad de mapear visualmente objetos de tu aplicación a tablas de una base de datos relacional. Esto se logra a través de diagramas intuitivos que muestran las relaciones entre las clases y las tablas.

OpenAccess se integra de manera estrecha con entornos de desarrollo integrados (IDEs) populares como Visual Studio y Eclipse.

Genera automáticamente las sentencias SQL necesarias para interactuar con la base de datos

Picolisp

Lo que distingue a PicoLisp es su enfoque simple y minimalista, su integración nativa con Lisp, la ausencia de SQL y su flexibilidad en el almacenamiento de objetos, lo que lo hace adecuado para aplicaciones pequeñas o embebidas que requieren un sistema de bases de datos ligero y eficiente.

PicoLisp combina características tanto de un lenguaje de programación de código abierto como de un sistema de gestión de bases de datos.

Se ejecuta en Linux y otros sistemas compatibles con POSIX.

Está construido sobre un único tipo de datos interno (celda).

FastObjects

FastObjects es un Sistema de Gestión de Bases de Datos Orientadas a Objetos (SGBDOO) desarrollado inicialmente por Poet Software y posteriormente adquirido por Versant. Está diseñado para aplicaciones embebidas y distribuidas que requieren alta eficiencia y capacidad para manejar datos complejos. Este sistema es particularmente adecuado para entornos empresariales y técnicos que necesitan persistencia de objetos sin la sobrecarga de un sistema relacional.

Entre sus características esenciales destacan su compatibilidad con lenguajes de programación orientados a objetos, como Java y C++, su soporte para persistencia transparente de objetos y su optimización para sistemas embebidos. FastObjects utiliza un modelo de licencia comercial, orientado a clientes empresariales que buscan soluciones escalables y específicas para sus necesidades. Aunque su adopción ha disminuido en favor de bases de datos híbridas o multimodelo, sigue siendo relevante en nichos especializados donde la orientación a objetos es clave.



Ozone

Ozone es un Sistema de Gestión de Bases de Datos Orientadas a Objetos (SGBDOO) de código abierto, diseñado para ser ligero y adaptable, especialmente para aplicaciones desarrolladas en Java. Este sistema permite una integración natural con el paradigma orientado a objetos, ofreciendo persistencia directa de objetos y eliminando la necesidad de mapeo relacional. Su naturaleza de código abierto, bajo la Licencia Pública General (GPL), lo hace accesible para proyectos experimentales, educativos o de pequeña escala.

Aunque su desarrollo se ha ralentizado con el tiempo, Ozone sigue siendo una opción útil en casos específicos, como proyectos académicos o aplicaciones pequeñas y medianas que no requieren sistemas relacionales complejos. Su enfoque en Java y su simplicidad lo convierten en una herramienta práctica para aprender y explorar conceptos de bases de datos orientadas a objetos, aunque su uso en la industria es limitado.

ObjectDatabase++

ObjectDatabase ++ (ODBPP) es una base de datos embebida orientada a objetos diseñada para aplicaciones de servidor que requieren un mantenimiento externo mínimo.

Está escrito en C ++.

Tiene la capacidad de recuperarse automáticamente de fallos del sistema mientras se mantiene la integridad de la base de datos.

Jasmine

Jasmine es un Sistema de Gestión de Bases de Datos Orientadas a Objetos (SGBDOO) desarrollado originalmente por Fujitsu en los años 90. Este sistema se destacó por su capacidad para manejar datos complejos y estructurados, combinando características avanzadas de orientación a objetos como herencia, encapsulación y polimorfismo. Jasmine fue diseñado para integrarse con aplicaciones multimedia, científicas y empresariales que requerían modelos de datos ricos y flexibles.

Entre sus características principales se encuentran su compatibilidad con lenguajes de programación orientados a objetos y su capacidad para gestionar datos jerárquicos y multimedia. Jasmine operaba bajo un modelo de licencia comercial, lo que lo hizo popular en aplicaciones empresariales específicas durante su apogeo. Sin embargo, su uso ha disminuido significativamente con la adopción de sistemas más modernos y con modelos híbridos o multimodelo, aunque sigue siendo reconocido por su impacto histórico en el desarrollo de los SGBDOO.

6. Comparativa entre tipos de B.D.

Cada tipo de bases de datos que hemos estudiado, tiene diferentes propiedades, hay que tener claro cuáles son sus diferencias para elegir la más adecuada para la gestión de la información en el programa que vayamos a realizar.

Es muy importante una buena elección del tipo de base de datos.



6.1. Entre las BDOO con las BDR

Características de las BDOO. Diferencias con las relacionales

- En una BDOO, los datos se almacenan como objetos en vez de como tablas.
- Los objetos son entidades que describen el estado (atributos) y el comportamiento (métodos).
- Los objetos se identifican mediante un identificador de objetos único (OID).
 - No son visibles por el usuario.
 - Son independientes del contenido.
 - Dos objetos iguales con diferente OID son equivalentes, pero son entidades diferentes.
- Encapsulamiento: cada objeto contiene y define procedimientos (métodos) y la interfaz mediante la cual se puede acceder a él.
- Algunos de los conceptos de programación orientada a objetos utilizados son:
 - Clase.
 - Herencia.
 - Polimorfismo.
 - Sobrecarga.

Ventajas:

- **Mayor capacidad de modelado.** Se puede modelar el mundo real de una forma más fiel.
- **Construcción de nuevos tipos de datos a partir de los existentes.**
- **Menor redundancia.** Se puede reducir la redundancia agrupando propiedades comunes de diversas clases en una superclase.
- **Reusabilidad de clases.** Mantenimiento más sencillo y menos tiempo de desarrollo.
- **Mejor rendimiento.** Al menos en las aplicaciones de ingeniería. En las aplicaciones clásicas de bases de datos relacionales, éstas tienen mejor rendimiento.

Inconvenientes:

- Carencia de un modelo de datos universal.
- Falta experiencia.
- Hacen falta estándares.
- La optimización de consultas compromete el concepto de encapsulación.



6.2. Entre B.D (NoSQL) y las relacionales

Diferencias de bases NoSQL con las bases de datos relacionales

Algunas de las diferencias más destacables son:

- Arquitectura distribuida.
- No utilizan tablas para almacenar datos. Utilizan otros sistemas (clave-valor, grafos u objetos).
- No utilizan SQL como lenguaje de consultas. En ocasiones pueden utilizarlo como lenguaje de apoyo.
- No suelen permitir operaciones JOIN ya que la sobrecarga puede ser excesiva.

Ventajas de B.D NoSQL con las B.D. relacionales

Las ventajas más significativas son:

- No necesitan tantos recursos. Pueden ejecutarse en máquina de menos prestaciones.
- Escalabilidad horizontal: Para mejorar el rendimiento tan solo debemos añadir más nodos.
- Pueden manejar gran cantidad de datos.
- No genera cuellos de botella.

7. Bibliografía

- Fundamentos de bases de datos 4ª edición. Silberschatz, Korth, Sudarshan. Editorial McGraw-Hill.
- <http://informatica.uv.es/docencia/biblioguia/BD/ficheros/tema2.pdf>.
- <https://www.mindmeister.com/es/1079684487/las-12-reglas-de-codd-del-modelo-relacional>.
- <https://es.slideshare.net/victorquintero52/reglas-de-codd-47386443>.
- <http://www.ugr.es/~eues/webgrupo/Docencia/TovarDiaz/InfGestionII/tema5.pdf>.
- <https://es.wikipedia.org>.
- <https://en.wikipedia.org>.
- <http://es.tldp.org/Postgresql-es/web/navegable/tutorial/x574.html>.



- <https://revistadigital.inesem.es/informatica-y-tics/los-gestores-de-bases-de-datos-mas-usados/>.
- <https://bdooinfo.wordpress.com/sistema-gestor-de-bases-de-datos-orientados-a-objeto-sgbdoo/>.
- <https://es.slideshare.net/PabloTlv/sgbdoo-13572292>.
- <https://www.acens.com/wp-content/images/2014/02/bbdd-nosql-wp-acens.pdf>.
- <https://www.genbeta.com/desarrollo/bases-de-datos-nosql-elige-la-opcion-que-mejor-se-adapte-a-tus-necesidades>.
- Bases de datos NoSQL. Fuente:
<https://upload.wikimedia.org/wikipedia/commons/c/c1/Nosql.gif>.
- Piluca Tomás Escobar. Técnico de Hardware, Software, Redes y Programación.
- Logo DynamoDB.
<https://commons.wikimedia.org/wiki/File:DynamoDB.png>.
- Logo Casandra.
https://commons.wikimedia.org/wiki/File:Cassandra_logo.svg.
- Logo Redis https://en.wikipedia.org/wiki/File:Redis_Logo.svg.
- Logo MongoDB <http://www.mongodb.com>.
- Logo CouchDB <http://couchdb.apache.org>.
- Logo Neo4J <https://neo4j.com>.

